Data Structures and Algorithms – IT1170

Practical 4 – Queues / Answers

Exercise 1

- Q1 - Link Class

Store:

- Student ID
- Marks
- Next reference

```
class Link {

    int studentId;
    int marks;

    Link next;

    public Link(int studentId, int marks) {

        this.studentId = studentId;
        this.marks = marks;
        this.next = null;
    }

    public void displayLink() {

        System.out.println(
            "Student ID: " + studentId +
            ", Marks: " + marks);
    }
}
```

- Q2 - LinkedList Class

```
class LinkedList {

    private Link first;

    public LinkedList() {
```

```
        first = null;
    }

    public boolean isEmpty() {

        return first == null;
    }
}
```

- Q3 - Required Methods

InsertFirst()

```
public void insertFirst(int id, int marks) {

    Link newLink = new Link(id, marks);

    newLink.next = first;

    first = newLink;
}
```

Find()

```
public Link find(int key) {

    Link current = first;

    while(current != null) {

        if(current.studentId == key) {
            return current;
        }

        current = current.next;
    }

    return null;
}
```

InsertAfter()

```
public boolean insertAfter(
        int key,
        int id,
        int marks) {
```

```
    Link current = first;

    while(current != null &&
          current.studentId != key) {

        current = current.next;
    }

    if(current == null) {
        return false;
    }

    Link newLink =
            new Link(id, marks);

    newLink.next = current.next;

    current.next = newLink;

    return true;
}
```

DeleteFirst()

```
public Link deleteFirst() {

    if(isEmpty()) {
        return null;
    }

    Link temp = first;

    first = first.next;

    return temp;
}
```

Delete()

```
public Link delete(int key) {

    Link current = first;
    Link previous = first;

    while(current != null &&
          current.studentId != key) {
```

```
            previous = current;
            current = current.next;
        }

        if(current == null) {
            return null;
        }

        if(current == first) {

            first = first.next;
        }
        else {

            previous.next = current.next;
        }

        return current;
    }
```

Display List

```
    public void displayList() {

        Link current = first;

        while(current != null) {

            current.displayLink();

            current = current.next;
        }
    }
```

- Q4 - Main Method

```
public class Main {

    public static void main(String[] args) {

        LinkedList list =
                new LinkedList();

        list.insertFirst(101,80);
        list.insertFirst(102,75);
        list.insertFirst(103,90);
```

```
        System.out.println("Students:");

        list.displayList();

        System.out.println(
                "\nFinding Student 102");

        Link found =
                list.find(102);

        if(found != null)
            found.displayLink();

        list.delete(102);

        System.out.println(
                "\nAfter Delete:");

        list.displayList();
    }
}
```

Exercise 2

- Q5 - Book Link Class

```
class BookLink {

    int bookId;
    String title;
    int copies;

    BookLink next;

    public BookLink(
            int bookId,
            String title,
            int copies) {

        this.bookId = bookId;
        this.title = title;
        this.copies = copies;

        next = null;
    }

    public void displayLink() {

        System.out.println(
            bookId + " "
            + title + " "
            + copies);
    }
}
```

- Q6 - Linked List Methods

InsertFirst

```
public void insertFirst(
        int id,
        String title,
        int copies)
{
    BookLink newLink =
            new BookLink(
                    id,
                    title,
```

```
                    copies);

    newLink.next = first;

    first = newLink;
}
```

Find

```
public BookLink find(int key)
{
    BookLink current = first;

    while(current != null)
    {
        if(current.bookId == key)
            return current;

        current = current.next;
    }

    return null;
}
```

InsertAfter

```
public boolean insertAfter(
        int key,
        int id,
        String title,
        int copies)
{
    BookLink current = first;

    while(current != null &&
          current.bookId != key)
    {
        current = current.next;
    }

    if(current == null)
        return false;

    BookLink newLink =
            new BookLink(
                    id,
                    title,
```

```
                    copies);

    newLink.next = current.next;

    current.next = newLink;

    return true;
}
```

DeleteFirst

```
public BookLink deleteFirst()
{
    if(first == null)
        return null;

    BookLink temp = first;

    first = first.next;

    return temp;
}
```

Delete

```
public BookLink delete(int key)
{
    BookLink current = first;
    BookLink previous = first;

    while(current != null &&
          current.bookId != key)
    {
        previous = current;
        current = current.next;
    }

    if(current == null)
        return null;

    if(current == first)
        first = first.next;
    else
        previous.next =
                current.next;
```

```
    return current;
}
```

- Q7 - Main Program

Scenario Implementation

```
public class LibraryTest {

    public static void main(String[] args) {

        LibraryList library =
                new LibraryList();

        // Insert Initial Books

        library.insertFirst(
                101,
                "Java",
                5);

        library.insertFirst(
                102,
                "DSA",
                3);

        library.insertFirst(
                103,
                "Database",
                4);

        // Add after specific book

        library.insertAfter(
                102,
                104,
                "Networks",
                2);

        // Find

        BookLink book =
                library.find(103);

        if(book != null) {
```

```
            System.out.println(
                    "Book Found:");

            book.displayLink();
        }

        // Remove Book

        library.delete(102);

        // Delete First

        library.deleteFirst();

        // Display Books

        System.out.println(
                "\nAvailable Books:");

        library.displayList();
    }
}
```